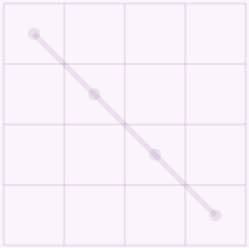
11

Лекция 20–21 апреля

Проблема остановки

Константин Чухарев

Дискретная математика — Весна 2027



Диагональный аргумент

> *Сущность математики лежит именно в её свободе.*
>
> ***— Георг Кантор***

Диагональ Кантора

Диагональный метод Кантора доказывает несчётность действительных чисел.

Пусть все действительные числа выписаны в список $r_1, r_2, r_3, \ldots$.

Новое число строится по диагонали, его n-я цифра отлична от n-й цифры числа r_n.

Новое число отличается от каждого элемента списка в своей позиции.

Значит, списка всех действительных чисел не существует.

> Тот же приём убьёт мечту об универсальном анализаторе программ.
>
> Диагонализация везде устроена одинаково. Объект, отличающийся от любого элемента перечисления, обязан в нём состоять.

Инверсия диагонали

Все машины выстраиваются в список $M_1, M_2, M_3, \ldots$, поскольку каждая задаётся конечным текстом.

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$
M_1	ост.	ост.	цикл	ост.
M_2	цикл	цикл	ост.	ост.
M_3	ост.	цикл	цикл	цикл
M_4	ост.	ост.	ост.	цикл
D — инверсия	цикл	ост.	ост.	ост.

Строка D — инверсия диагонали. На коде $\langle M_i \rangle$ она делает противоположное M_i.

Но D тоже машина, поэтому она занимает в списке некоторую позицию j. В собственном столбце строка D даёт противоположное тому, что машина D делает на этом слове.

Самоприменимость машин

Кодировка делает машину объектом того же вида, что и вход.

> Кодировка $\langle M \rangle$ — такая же строка, как любая другая, а машина умеет читать строки.
>
> Подать машине её собственное описание — не страннее, чем передать функции её номер.

После универсальной машины это уже не сюрприз. На паре $\langle M \rangle \langle M \rangle$ универсальная машина просто симулирует M на её собственном коде, и отдельного механизма для этого не нужно.

Диагонализация требует лишь одного — выразительности, достаточной для разговора о себе.

Проблема остановки

> *Математику можно определить как предмет, в котором мы никогда не знаем ни того, о чём говорим, ни того, истинно ли то, что мы говорим.*
>
> ***— Бертран Рассел***

Язык HALT

Формально проблема остановки — это язык пар из описания машины и слова.

Определение 1 (HALT)

Нужно по паре $(\langle M \rangle, w)$ узнать, останавливается ли M на входе w.

$$\mathrm{HALT} = \{\langle M \rangle w \mid M \text{ останавливается на } w\}.$$

Пример (Конкретные запросы)

- M_{loop} на любом входе сразу зацикливается, поэтому $\langle M_{\text{loop}} \rangle 0 \notin \mathrm{HALT}$.
- M_{acc} на любом входе сразу принимает, поэтому $\langle M_{\text{acc}} \rangle \varepsilon \in \mathrm{HALT}$.
- M_1 принимает слово с первым символом 1, а на 0 уходит в цикл, поэтому пара $(\langle M_1 \rangle, 1)$ лежит в HALT, а пара $(\langle M_1 \rangle, 0)$ — нет.

Описание $\langle M_1 \rangle$ с одним словом даёт пару из HALT, с другим — пару вне его. Вопрос в одном алгоритме, который отвечает на все пары сразу.

Почему симуляция не решает?

Наивный план состоит в том, чтобы запустить U на $\langle M \rangle w$ и подождать ответа.

Если M останавливается — симуляция увидит это.

Если M зациклилась, ждать можно вечно.

Любой конечный таймаут произволен. Зациклившаяся машина может остановиться на секунду позже.

Симулятор лишь воспроизводит проблему — нужен другой решатель, и его не существует.

Неразрешимость остановки

Препятствие не техническое. Утверждение Тьюринга говорит, что такого объекта нет вовсе.

Теорема 1

Не существует МТ, которая по паре $(\langle M \rangle, w)$ всегда останавливается и решает, остановится ли M на w.

Доказательство идёт от противного в четыре шага.

1. допустить, что решатель H существует
2. собрать машину-обёртку D над текстом H
3. подставить D в неё саму
4. получить противоречие разбором случаев

Гипотетический решатель

Теорема запрещает машину с тремя обещаниями сразу.

Определение 2 (Решатель HALT)

МТ H называется решателем языка HALT, если на входе $\langle M \rangle w$ она

- всегда останавливается
- принимает, когда M останавливается на w
- отвергает, когда M зацикливается на w

Допущение выглядит скромно. Машина H задана конечным текстом программы.

Этот текст можно скопировать и встроить в другую машину как подпрограмму, и на этой возможности держится доказательство.

Машина-обёртка

Диагональ требует машины, которая ведёт себя противоположно решателю.

Пример (Программа, построенная из программы)

Имея текст H, собираем машину D.

1. прочитать вход — описание $\langle M \rangle$
2. составить строку $\langle M \rangle \langle M \rangle$
3. вызвать H на этой строке
4. при приёме H — уйти в бесконечный цикл
5. при отвержении H — остановиться

D — законная машина Тьюринга. Встраивание готовой машины как подпрограммы мы уже делали при строительстве универсальной машины.

Смысл обёртки — инверсия. На вопрос о паре $(\langle M \rangle, \langle M \rangle)$ машина D отвечает противоположно решателю H.

Подстановка машины в себя

Машина D готова работать с любым описанием на входе — подадим ей собственное, $\langle D \rangle$.

На входе $\langle D \rangle$ верны сразу два утверждения, каждое по своему определению.

Контракт решателя на паре $(\langle D \rangle, \langle D \rangle)$ таков.

$$H(\langle D \rangle \langle D \rangle) \text{ принимает} \leftrightarrow D \text{ останавливается на } \langle D \rangle$$

Код обёртки на входе $\langle D \rangle$ даёт обратное.

$$D \text{ останавливается на } \langle D \rangle \leftrightarrow H(\langle D \rangle \langle D \rangle) \text{ отвергает}$$

Собираем цепочку. Приём H равносилен остановке D, а остановка D — отвержению H.

Противоречие

Доказательство

Докажем разбором случаев поведение D на входе $\langle D \rangle$.

Случай 1. D останавливается на $\langle D \rangle$. Тогда по контракту H на паре $(\langle D \rangle, \langle D \rangle)$ принимает. По коду обёртки приём H переводит D в бесконечный цикл — вопреки остановке.

Случай 2. D зацикливается на $\langle D \rangle$. Тогда H на той же паре отвергает. По коду обёртки отвержение заставляет D остановиться — вопреки циклу.

Любое поведение D опровергает контракт H, поэтому решателя HALT не существует.

Где именно ломается допущение?

Противоречие бьёт не по машине D, а по предположению о решателе.

Машина не обязана останавливаться

Противоречия внутри D нет. Машина имеет право зацикливаться, а обязанность всегда останавливаться была только у H.

Ломается допущение. При существующем H машина D обязана на $\langle D \rangle$ и остановиться, и зациклиться.

Распознаваема, но не разрешима

Диагональ запретила только решатель.

Распознаватель HALT получается симуляцией. Универсальная машина повторяет работу M на w и принимает при остановке. На паре с зацикливанием зацикливается и она.

Приём совпадает с ответом «да», а отвергать распознаватель не обязан.

Разница между классами ровно в обещании остановки на словах вне языка.

Распознаваемость с двух сторон

Две распознавательские машины собираются в одну решающую.

Теорема 2 (Критерий разрешимости)

Язык L разрешим тогда и только тогда, когда распознаваемы и L, и $\overline{L}$.

Доказательство

Докажем эквивалентность в обе стороны. Если L разрешим, его разрешитель распознаёт L, а инверсия его ответа распознаёт $\overline{L}$. Обратно, распознаватели R и S идут чередующимися шагами, по одному шагу за такт. Слово лежит ровно в одном из двух языков, поэтому одна из машин рано или поздно примет, и процедура остановится. Приём R даёт ответ «да», приём S — ответ «нет».

Критерий превращает нераспознаваемость дополнения в неразрешимость языка.

Замыкание распознаваемых языков

Распознаватель обещает приём своих слов и ничего больше.

Теорема 3

Распознаваемые языки замкнуты относительно объединения и пересечения, но не дополнения.

Доказательство

Объединение. Распознаватели идут чередующимися шагами, составной принимает, когда приняла любая из машин.

Пересечение. Машины идут последовательно, составной принимает, только если приняли обе. Зацикливание одной из них здесь законно.

Дополнение. Замкнутости нет. Например, дополнение HALT — пары с зацикливанием, и распознавателя у него нет, иначе критерий дал бы решатель HALT.

Замыкание разрешимых языков

Разрешители останавливаются на любом входе.

Теорема 4

Разрешимые языки замкнуты относительно объединения, пересечения и дополнения.

Доказательство

Объединение. Разрешители идут по очереди, приём по первому приёму.

Пересечение. Тот же запуск, приём только после приёма обеих.

Дополнение. Меняем ответ на противоположный, остановка гарантирована.

Последовательный запуск годится всюду, потому что решатель не зацикливается.

Одно семейство доказательств

Объект	Самореференция	Полученное «нет»
Лжец	$S \leftrightarrow \neg S$	нет истинностного значения
Рассел	$R = \{x \mid x \notin x\}$	нет множества
Кантор	диагональ вне списка	нет перечисления
Гёдель	«недоказуемо»	истина вне вывода
Тьюринг	D на своём коде	нет алгоритма

Система, достаточно выразительная, чтобы говорить о себе, порождает неразрешимость.

Запрет держится на выразительности системы, и новый алгоритм его не снимет.

Сведения

> *Если задачу решить не удаётся, рядом есть более простая задача, которую решить можно. Найдите её.*
>
> ***— Дьёрдь Пойа***

Сведение

Умение решать B переносится на A, если вход A пересчитывается во вход B.

Определение 3 (Сведение (*many-one reduction*))

$A \underset{m}{\leq} B$, если существует вычислимая тотальная функция f, для которой

$$w \in A \leftrightarrow f(w) \in B.$$

Решив B, решаем и A. Считаем $f(w)$ и спрашиваем решатель B.

Сведение переносит алгоритмы — и неразрешимости.

Как пользоваться сведением?

Вопрос о w переводится в вопрос о $f(w)$.

Теорема 5 (Логика переноса)

- Если $A \leq_m B$ и B разрешима, то разрешима и A.
- По контрапозиции, если $A \leq_m B$ и A неразрешима, то неразрешима и B.

Доказательство

Пусть f — функция сведения, R — разрешитель B.

На входе w машина вычисляет $f(w)$, запускает R и выдаёт его ответ — по свойству сведения это ответ на вопрос $w \in A$.

Оба шага останавливаются на каждом входе, поэтому A разрешима, а контрапозиция даёт второй пункт.

Направление сведения

Сведение через контрапозицию превращается в инструмент доказательства неразрешимости. Направление выбирается один раз. Ошибка в направлении делает весь вывод неверным. Каждый пример ниже следует этому правилу — от HALT к новой задаче.

Сведение к разрешимой задаче доказывает только разрешимость.

Чтобы доказать неразрешимость задачи X, сводят к ней известную неразрешимую, то есть строят $\mathrm{HALT} \underset{m}{\leq} X$.

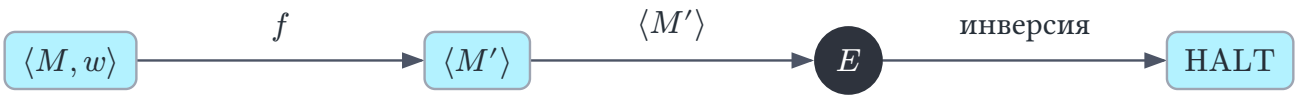
Пустота языка

Пример (От HALT к E_{TM})

$E_{\text{TM}} = \{\langle M \rangle \mid L(M) = \varnothing\}$ — описания машин с пустым языком.

Строим сведение $f(\langle M \rangle, w) = \langle M' \rangle$, где M' игнорирует свой вход, симулирует M на w и принимает при остановке.

M остановилась — язык Σ^*, зациклилась — язык $\varnothing$.



Разрешитель E_{TM} с инверсией ответа решал бы HALT — значит, E_{TM} неразрешима.

Остановка на пустом входе

Пример ($\mathrm{HALT} \underset{m}{\leq} \mathrm{HALT}_\varepsilon$)

Остановка на пустом входе образует язык $\mathrm{HALT}_\varepsilon = \{\langle M \rangle \mid$ M останавливается на пустом входе$\}$.

Обёртка $f(\langle M \rangle, w) = \langle M' \rangle$ задаёт машину M', которая на любом входе делает следующее.

1. стирает ленту и записывает на неё строку w
2. возвращает головку к началу записи
3. дальше исполняет таблицу переходов машины M

На пустом входе M' сама готовит себе строку w — дальше машины неотличимы.

Тогда M останавливается на $w \leftrightarrow M'$ останавливается на ε.

Значит, $\mathrm{HALT}_\varepsilon$ неразрешима.

Сведение к эквивалентности программ

Пример ($\mathrm{HALT} \underset{m}{\leq} \mathrm{EQ}_{\mathrm{TM}}$)

Пары машин с равными языками образуют язык $\mathrm{EQ}_{\mathrm{TM}} = \{\langle M_1 \rangle \langle M_2 \rangle \mid L(M_1) = L(M_2)\}$.

Обёртка $f(\langle M \rangle, w) = \langle M' \rangle \langle A \rangle$ состоит из двух машин.

- M' игнорирует свой вход, симулирует M на w и принимает, если симуляция остановилась
- A принимает любой вход сразу

M остановилась на $w \Rightarrow L(M') = \Sigma^* = L(A)$ — машины эквивалентны.

M зациклилась $\Rightarrow L(M') = \varnothing \neq L(A)$ — машины различны.

Ответ «да» решателя $\mathrm{EQ}_{\mathrm{TM}}$ — сразу ответ «да» для HALT, инверсия не нужна.

Сведение к «печатает 42»

Пример ($\mathrm{HALT} \leq_m \mathrm{PRINT}_{42}$)

Программы, которые когда-нибудь печатают 42, образуют язык $\mathrm{PRINT}_{42} = \{\langle P\rangle \mid P$ когда-нибудь печатает $42\}$.

Обёртка $f(\langle M\rangle, w) = \langle P\rangle$ строит программу P, работающую на пустом входе.

1. симулирует M на входе w
2. если симуляция остановилась — печатает 42

Пока симуляция идёт, вывод P пуст. Поэтому P напечатает 42 тогда и только тогда, когда M остановится на w.

Три сведения собраны одной конструкцией. Обёртка вшивает вычисление M на w, а ответ читается по одному наблюдаемому свойству обёртки. Свойства разные — пустота языка, равенство языков, появление 42 в выводе.

Программа анализирует программу

Антивирус хочет решить, опасна ли эта программа.

Статический анализатор хочет решить, содержит ли код утечку.

Верификатор хочет решить, удовлетворяет ли программа спецификации.

> Все три вопроса заданы поведением программы на входах.
>
> К точному ответу на каждый из них сводится HALT, поэтому точного решателя нет.

Верификация работает в разрешимых фрагментах, и вне них обещания полноты нет.

Теорема Райса формулирует тот же запрет сразу для любого нетривиального свойства языка.

Другие неразрешимые задачи

Пример (Семейство от HALT)

- **Тотальность** — останавливается ли M на каждом входе?
- **Эквивалентность** — верно ли $L(M_1) = L(M_2)$?
- **Регулярность** — является ли $L(M)$ регулярным?
- **Контекстная свобода** — является ли $L(M)$ контекстно-свободным?

Для тотальности сведение строит машину M', которая игнорирует свой вход и симулирует M на w. Поведение M' одинаково на всех входах, поэтому она тотальна тогда и только тогда, когда M останавливается на w.

Тотальность спрашивает о поведении машины на всех входах, остальные три — о свойствах её языка.

Что остаётся инженеру?

Запрещена полнота обещания, инструменты при этом работают. Идеального отладчика и универсального доказателя завершения не будет.

Каждый практический инструмент спасается одним из трёх ходов.

1. сузить класс программ, о которых спрашивают
2. оставить инструменту право ответить «не знаю»
3. заменить программу моделью и отвечать о модели

Выбор хода — инженерное решение с известной ценой. Чем шире класс, тем грубее ответ.

Инструменты и их честная рамка

Инструмент	Обещание	Честная рамка
Отладчик	найти любое зависание	один прогон — вечный цикл выглядит как долгая работа
Анализатор завершения	доказать завершение	правила для узких классов циклов — вне класса «не знаю»
Линтер	поймать все дефекты	заданные паттерны — срабатывания и пропуски неизбежны
Система типов	принять всё корректное	разрешимый фрагмент — отвергает часть корректных
Проверка моделей (*model checking*)	проверить сам код	проверяет конечную модель, не сам код

Неразрешимость и остановка

Диагональ Кантора переехала на программы. Список машин вместе с ответами на собственных кодах противоречив. Отсюда неразрешимость остановки, а сведение переносит её на другие задачи. Дополнение разделяет классы, и у распознаваемых языков оно не замыкается.

- Диагональ даёт машину, которой нет в перечислении машин.
- HALT распознаваема симуляцией, но неразрешима, потому что D ломает любой решатель.
- Сведение переносит неразрешимость на пустоту, тотальность, эквивалентность и регулярность.
- Разрешимость — распознаваемость с двух сторон.
- Инструменты анализа программ сужают класс, отвечают «не знаю» или анализируют модель.

Вопросы по неразрешимости

о проблеме остановки

1. Воспроизведите схему диагонализации Кантора и укажите, где в ней живёт инверсия.
2. Проведите доказательство неразрешимости и объясните, что делает D на входе $\langle D \rangle$ в обоих случаях.
3. Почему HALT распознаваема, а её дополнение — нет?
4. Сформулируйте критерий разрешимости и объясните, почему чередование шагов двух распознавателей всегда заканчивается.
5. Определите сведение $A \leq_m B$ и объясните, в какую сторону его строить для доказательства неразрешимости.
6. Опишите обёртку в сведении от HALT к задаче «программа печатает 42».
7. Опишите сведение от HALT к проблеме пустоты языка.

Что почитать?

Задача	Где читать
Диагональ и неразрешимость HALT	m26, раздел «Проблема остановки»
Распознаваемость и замкнутость	m26, раздел «Разрешимость»
Строить сведения	m26, раздел «Сведения»